University of Minnesota custom Azure course

Lab 2 Solution: Archival Landing Zone

Instructor solution and facilitator checks

Session 2: Long-Term Archival, Data Movement at Scale, and Hybrid Identity

Session: Day 1, Session 2, Block 4

Audience: Instructor during delivery; attendees after the course

Includes: Expected output for every checkpoint, the as-code solution, debrief model answers, failure modes

Keep this document out of the participant folder until the course ends.

Contents

1. How to use this solution 3
2. Part 0 and Part 1: account with the security baseline 3
3. Part 2: immutable container, tagged upload, manifest 3
4. Part 3: rehydration 4
5. Part 4: lifecycle rule 4
6. Part 5: ABAC 5
7. Part 6: validation and cleanup 5
8. The as-code solution 6
9. Debrief: model answers 6
10. Dry-run failure modes 7

1. How to use this solution

This document is the instructor's companion to Lab 2. It contains the complete command sequence with expected output, the as-code deployment that produces the same end state in one step, the facilitator checks for every checkpoint, model answers for the debrief, and the failure modes seen in dry runs. Share it with attendees after the course if you wish; it is written so that it can stand alone.

> **Two ways to reach the end state**
>
> **Step by step** (what attendees do): sections 2 to 7 mirror the participant guide with the exact expected output.
>
> **As code** (for verification or a fast rebuild): `lab-solutions/s02-archival-landing-zone/deploy.sh` deploys `main.bicep`, seeds the dataset, proves the policy, and starts the rehydration in about three minutes. Section 8 explains the Bicep and how to run it.

2. Part 0 and Part 1: account with the security baseline

Expected output of Checkpoint 1:

```
Sku           Tls     PublicBlob    SharedKey    Hns
------------  ------  ------------  -----------  ------
Standard_ZRS  TLS1_2  False         False        False
true
```

If `SharedKey` shows `True`, the attendee missed `--allow-shared-key-access false`; fix with `az storage account update -n $SA -g $RG --allow-shared-key-access false`. If versioning shows `false`, the `blob-service-properties update` did not run; it is required for Part 2 only in the sense that the course pattern expects it (version-level WORM needs it); the container policy itself does not.

Common mistake: enabling last access tracking from habit. The immutability policy in Part 2 then fails with a message that the feature cannot be enabled on an account with last access tracking (or the reverse order fails). Disable it: `az storage account blob-service-properties update -n $SA -g $RG --enable-last-access-tracking false`.

3. Part 2: immutable container, tagged upload, manifest

Expected output of the policy create:

```
AllowProtectedAppendWrites    Etag
ImmutabilityPeriodSinceCreationInDays    State
----------------------------  ------------------------------------  ----------------------
-----------------  --------
False                         "8d0a1c..."                           1
Unlocked
```

Expected Checkpoint 2 delete error (first line is what attendees record):

```
The requested operation is not allowed as the blob is immutable due to a policy.
RequestId:...
Time:...
ErrorCode:BlobImmutableDueToPolicy
```

Facilitator check: `az storage blob list --account-name $SA -c satellite-archive --auth-mode login --include t --query "length([?tags.RetentionClass=='Permanent'])"` returns 12 (13 after the manifest is counted if they uploaded it to the same container by mistake; that is harmless).

Why the broad data role is granted and later removed

Attendees are account Contributors, which carries no data-plane rights under Microsoft Entra authorization. Storage Blob Data Contributor is granted for the upload and removed in Part 5 so that the conditional Reader role is the only data role in effect. If an attendee skips the removal, Part 5's CISM download succeeds (RBAC is additive) and the lesson is lost; ask them to run the delete and retest.

4. Part 3: rehydration

Expected after the request:

```
Tier     ArchiveStatus               Priority
-------  --------------------------  ----------
Archive  rehydrate-pending-to-cool   Standard
```

Typical observed completion in dry runs for a 2 MB object at Standard priority: between 2 and 9 hours. Collect the room's numbers the next morning; the spread is the point. If someone raises the priority with `--rehydrate-priority High` on a pending request, the service accepts the change (Standard can be raised to High while pending; the reverse is not allowed).

5. Part 4: lifecycle rule

Expected output of Checkpoint 4:

```
Name                              Enabled    Tag
--------------------------------  ---------  ---------
permanent-to-archive-by-creation  True       Permanent
```

If an attendee asks why the rule did nothing during the lab: Azure Storage runs lifecycle policies once per day; the first run after policy creation can take up to 24 hours. Tomorrow morning, `az storage blob list ... --query "[].{name:name,tier:properties.blobTier}"` shows the eleven online objects in Archive (the twelfth is already there). Note that the rehydrating object is excluded from tiering until rehydration completes.

6. Part 5: ABAC

The condition, exactly as the Microsoft Learn example is written (with the `Blob.List` exclusion so listing still works):

```
((!(ActionMatches{'Microsoft.Storage/storageAccounts/blobServices/containers/blobs/read'} AND NOT
SubOperationMatches{'Blob.List'})) OR
(@Resource[Microsoft.Storage/storageAccounts/blobServices/containers/blobs/tags:ResearchUnit<$key_case_sensi
StringEquals 'PGRL'))
```

Expected validation results:

Command	Expected result	Why
`az storage blob list` on satellite-archive	Succeeds	The condition excludes the Blob.List suboperation
Download `noaa19_pass_2026_10_01.bin`	Succeeds	Tag ResearchUnit equals PGRL
Download `cism-reference/cesm2_note.txt`	AuthorizationPermissionMismatch	Tag ResearchUnit equals CISM, condition false
Upload to `manifests`	AuthorizationPermissionMismatch	Storage Blob Data Reader has no write action

Propagation: data-plane evaluation of a new assignment can lag by a few minutes. Dry runs needed 30 to 120 seconds. The participant guide builds in a 60-second pause; if the PGRL download fails at the first try, waiting is the fix, not re-creating the assignment (which creates a duplicate with a different GUID).

Model answers for 7.4: a single condition on the research unit tag replaces every per-share and per-folder ACL for that unit; tags must be applied at ingest by the pipeline or the upload tool (the lab used `--tags` on upload), and a second condition on `@Request` tags can require that new blobs carry the tag, which is the next step in production.

7. Part 6: validation and cleanup

Expected verifier summary (11 matched, 1 offline):

```
{
  "datasetId": "DS-LAB02-ajs07",
  "manifestHashValid": true,
  "objectsChecked": 12,
  "matched": 11,
  "mismatched": [],
  "missing": [],
  "offlineArchiveTier": ["noaa19_pass_2026_10_12.bin"],
  "result": "PASS"
}
```

Cleanup order matters: `immutability-policy delete --if-match "*"` first (works only because the policy is unlocked), then `container-rm delete` (management plane), then the account. If an attendee locked the policy against the instructions, the container cannot be deleted until the one-day interval passes; the resource group deletion will fail until then. Tell them to leave it for tomorrow.

8. The as-code solution

`lab-solutions/s02-archival-landing-zone/main.bicep` produces the Part 1, 2 (container and policy), 4, and 5 (role assignment with condition) end state in one deployment. Uploads, the rehydration request, and the validation are data-plane operations handled by `deploy.sh`.

```
export LAB_RG=rg-lab-07 LAB_SUFFIX=ajs07
export LAB_USER_OBJECT_ID=$(az ad signed-in-user show --query id -o tsv)
cd lab-solutions/s02-archival-landing-zone
./deploy.sh
```

What the Bicep does, module by module:

Module	Resources	Notes
`modules/storage-account.bicep`	StorageV2 ZRS account, blob service properties, `manifests` container	Security baseline fixed in the module; last access tracking is a parameter defaulting to false
`modules/immutable-container.bicep`	`satellite-archive` container plus `immutabilityPolicies/default`	Created unlocked; locking needs the ETag and is a CLI step by design
`modules/lifecycle-policy.bicep`	`managementPolicies/default` with the tag-targeted rule	The policy is written in full every deployment
`modules/role-assignment.bicep`	Role assignment with `condition` and `conditionVersion: '2.0'`	GUID name derived from scope, principal, role, and condition, so re-deploys are idempotent

Verify an attendee's work with what-if: `az deployment group what-if -g rg-lab-07 --template-file main.bicep --parameters main.bicepparam`. Resources that match show as `NoChange`; anything the attendee missed shows as `Create`.

9. Debrief: model answers

Question	Representative answer	What to listen for
Where did the platform enforce a guarantee you rely on process for today?	The retention policy refused a delete from an account Contributor; today a deletion is prevented by backup tape rotation procedures, change control, or 'nobody has the password'. The ABAC condition enforced who may read; today it is ACL hygiene and periodic reviews.	Naming a specific control and a specific process; recognizing that process controls fail silently
How long did rehydration take, and what does that mean for researcher expectations?	Hours, not minutes, at Standard priority; High priority is faster and costs more. Researchers need a request process with an expected turnaround, and frequently requested archives belong in Cold, not Archive.	The connection to tier choice and to the Session 1 small-object guidance
Which ABAC condition would replace the most ACLs?	A read condition on a research unit or project tag at account scope, or a container-name condition for departmental shares; a request-tag condition that requires new uploads to carry the tag.	Awareness that tags must be governed at ingest
Why creation time instead of last access?	Last access tracking cannot be enabled on an account with immutable storage. Active research data belongs in a separate account without WORM, where last access tiering is available; this is the sharding-by-protection-class pattern from Session 1.	Linking the limitation to account boundaries
A locked policy at 36,500 days with the wrong scope?	The data (and the account) could not be deleted for 100 years; storage would be billed for the full period. Protection: test unlocked, validate scope and interval, lock within about a day, and treat the lock as a change-controlled decision owned by Security.	Understanding that locked means locked

10. Dry-run failure modes

Symptom	Cause	Fix
`upload-batch` fails with AuthorizationPermissionMismatch right after the role assignment	Propagation	Wait 60 seconds and retry; the guide includes a sleep
Policy create fails mentioning last access tracking	Tracking enabled in Part 1	Disable tracking, recreate the policy

Symptom	Cause	Fix
Checkpoint 2 delete succeeds	Policy created on the wrong container or not at all	`immutability-policy show` on `satellite-archive`; recreate
CISM download succeeds in Part 5	Broad data role still present, or the attendee is also in a group with a data role	`az role assignment list --assignee $ME --scope $SCOPE`; remove the extra role
`verify_manifest.py` import error	SDK packages missing	`pip install azure-identity azure-storage-blob`, or validate the local copy with `--dir satellite`
Resource group deletion fails the next day	A locked policy in the attendee's account	Wait for the interval; delete blobs, container, account, then the group